Recitation 8: Cyclic Association and Separate Compilation

Topics

- Cyclic Association
- Separate Compilation
- Namespaces

Attachments

- rec08_start.cpp
- rec08_output.txt
- Code Reading Questions

Overview

- This can be a big project, but we will break it into pieces, of hopefully easy steps.
- The overall goal is to implement a Registrar class that tracks Students and Courses. Students take courses and courses have students that are enrolled in them. And there we have **cycles**. When you are *all* done, you will have
    - solved the problem of cyclic association,
    - accomplished separate compilation,
    - and placed all of your classes and their code into a namespace.
- There are **four** checkouts:
    - First, solve the problem, putting the definitions for all methods and operators inside the classes, **except** where necessary. Cyclic association forces us to use prototypes for some methods / functions, putting their definitions somewhere after they are used. Which methods / functions does that apply to in this problem? Remember we have given you starter code... For this part, you will turn in a file called: **rec08_single_simple.cpp**. Do not define a namespace for this solution. Be sure to **get checked out** for this and upload to Brightspace.
    - Second, make a copy of this program. Call it **rec08_single.cpp**. Move the definitions of **all** the methods and definitions to the end of the file, i.e. after main. Again, be sure to **get checked out** for this and upload to Brightspace.

    - Third, again make a copy, and do all of the separate compilation. You do not need to do a separate upload for this.
    - Fourth, and finally, put your code in the namespace as specified below. Upload the files described below.
- Of course you want to test at every possible point! "Test early and often" is a common motto among successful developers.
- Only after your solution works in a single file (our second step above), then copy it and split it into multiple files, i.e. for each class a header file and an implementation file, along with the one cpp file for the "testing" code.
    - **Don't modify your original working code**, the solutions that are in a single file! Make copies.
    - When adding separate compilation, sometimes code that worked fine before now breaks. Good to have the working code to go back to and look at.
- **After doing separate compilation, get checked out again! Before adding the namespace!**
- Next **place the classes and the function / method definitions into the namespace BrooklynPoly**. (Don't do this for the single file version.)
- See below (at the bottom) for the complete list of files to turn in to Brightspace. But I think you know what they are.

Some Details

We have provided some starter code that is attached. It will save you a few minutes initially setting up the three classes and test code. No, it does not even begin to address the issues of cyclic association.

We are assuming that *names are unique* (if they weren't we would have to turn everyone into a number). Remember, just because we occasionally make such assumptions, as we are doing here, you should *never assume* names are unique unless your spec says so. I have noticed that there is more than one person with the name "John" in the real world. (Very confusing.)

Also, we are assuming that, miraculously, students' names are all single tokens (if they weren't, things would just be more annoying).

Where do you think the Course and Student objects should live? *Suppose* you just kept a vector of them, why might that prove to be a disaster? Will that work? No. You are going to have **pointers from a Course to the Students.** If the Students are in a vector, i.e. a vector<Student>, what might happen to their addresses with you push_back another Student? If the vector was full, then the underlying array would get reallocated and all the pointers to them **would become invalid**.

We are *not* asking you for copy control, despite the fact that the Registrar really *owns* all those objects on the heap. Sure, by now I expect you could do that "quickly", but you only have three hours.

Probably making a **method** out of the code that looks for the course or student by name would be useful. In fact we now have prototypes for such methods in the starter code. Don't hesitate to write other useful **methods**.

Remember to only pass to the methods what is needed. I have seen students pass a method the object's own fields. That's really "silly".

What to turn in

- **rec08_single_simple.cpp**: This will contain your **single file solution** with all methods / functions defined inside the classes, except for those that *have to be* defined outside, in order for the code to work.
- **rec08_single.cpp**: This will contain your **single file solution**.
- The seven files for separate compilation with the namespace.:
    - **rec08.cpp**: This should have just
        - your includes. Note you don't need includes here for course.h or student.h. (Do you see why?)
        - your using namespace declarations
        - `main`
    - **registrar.h**, **course.h**, **student.h**
    - **registrar.cpp**, **course.cpp**, **student.cpp**

Submit **Cancel**